MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 6

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Part 1: Read what xv6 shows you **3**

2 Part 2: Write mystat **5**

3 Part 3: Measure the limits **9**

4 Part 4: Write find **11**

Part 1: Read what xv6 shows you

Questions:

- What do the types 1, 2 and 3 mean (`kernel/stat.h`)?
- Which inode number does the root directory have, and why are `.` and `..` the same there?
- An empty directory `a` has size 32. A directory entry has a fixed size. How big is one entry, and which two entries does `a` contain?

Answers:

- The file `kernel/stat.h` defines three types: `T_DIR` is 1 (directory), `T_FILE` is 2 (regular file) and `T_DEVICE` is 3 (device). In Figure 1.1, `.` and `..` have the type 1, `README` and all the programs have the type 2, and `console` has the type 3. The result is summarized in Table 1. Unlike Linux, xv6 has no other types, in particular no symbolic links.
- The root directory has the inode number 1: in the first two lines of Figure 1.1 both `.` and `..` show the inode 1 and the size 1024. The root directory has no parent, so its entry `..` points to the directory itself, and for this reason the two entries are identical.
- One entry takes 16 bytes. The entry of a directory is a structure with the inode number `inum` (2 bytes) and a name of `DIRSIZ` = 14 bytes (Figure 3.1), so 2+14 = 16. The empty directory `a` contains only the two entries `.`, which is the directory itself, and `..`, which is its parent, the root directory. The size is therefore $2 \cdot 16 = 32$ bytes.

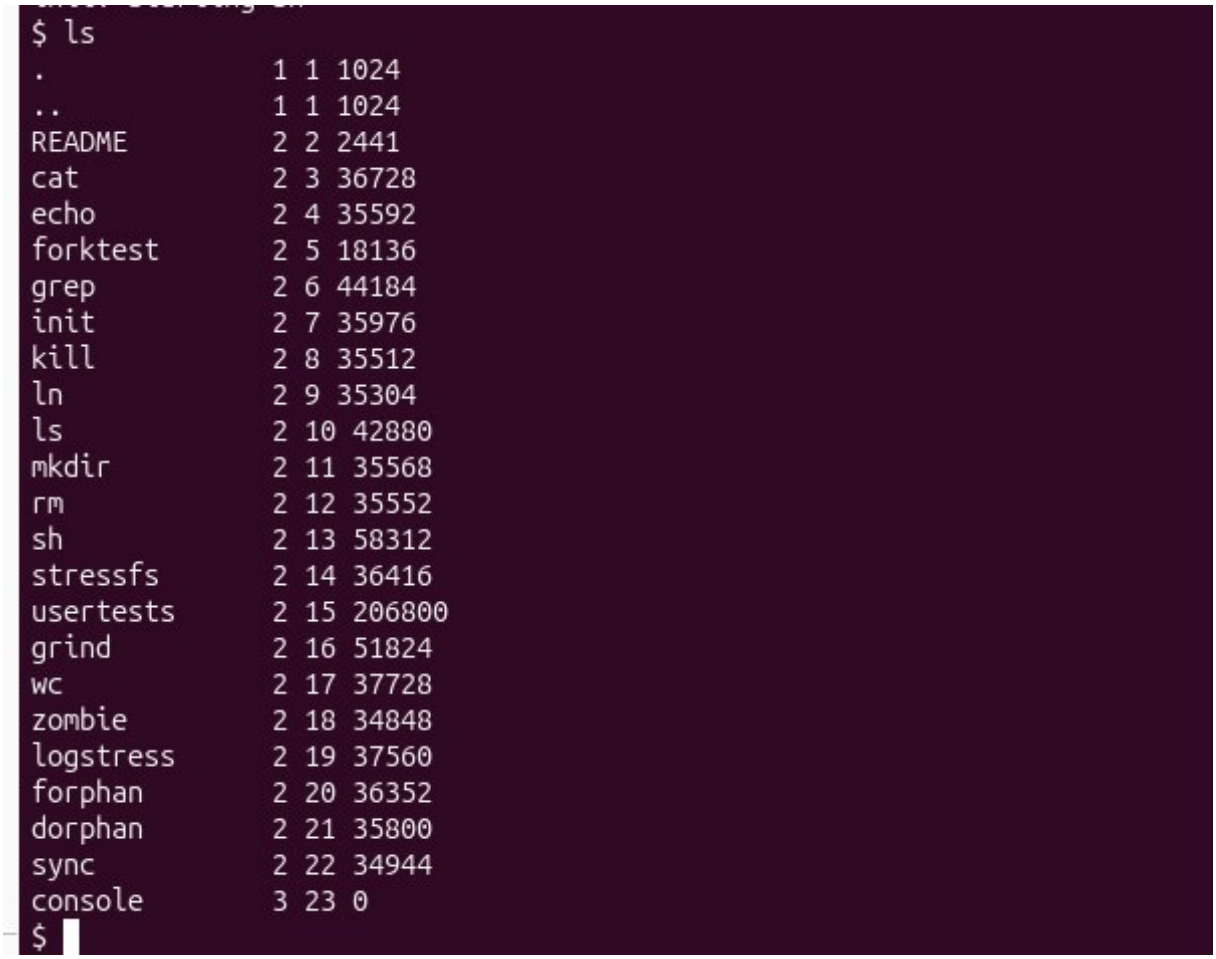


Figure 1.1: The output of `ls` in the xv6 shell

ls – in xv6 the program prints four columns for every entry: the name, the type, the inode number and the size in bytes. In Figure 1.1 the first two lines are `.` and `..` (type 1, inode 1, size 1024). They are followed by `README` (type 2, inode 2, 2441 bytes) and by the programs from `cat` (inode 3) to `sync` (inode 22), all of type 2; the biggest one is `usertests`, with 206800 bytes. The last line is `console`, with the type 3, the inode 23 and the size 0, because a device has no data in the file system. The inode numbers follow the order in which the image of the file system was built. The program `mystat` is not in the list yet, because it is written in Part 2.

mkdir a ; ls a – creates an empty directory and lists it. The listing contains only the entries `.` and `..`, and the size of the directory is 32 bytes, which agrees with the calculation in the answers.

Constant	**Value**	**Meaning**	**Example in `ls`**
`T_DIR`	1	directory	`.` and `..`
`T_FILE`	2	regular file	`README`, `cat`
`T_DEVICE`	3	device	`console`

Table 1: The file types of xv6

Table 1 shows that the file system of xv6 distinguishes only three kinds of objects, and that the type is stored in the inode and not in the name of the file.

Part 2: Write mystat

Questions:

- Paste the outputs after `ln` and after `rm`. What changed and what did not?
- Find the line in `kernel/sysfile.c` (`sys_unlink`) that decrements the link count. When are the data blocks freed?
- xv6 has no symbolic links and no permissions. Look at `struct dinode` in `kernel/fs.h`: which fields does an xv6 inode have?

Answers:

- After `ln README r2` (Figure 2.3) both names show the inode 2, the size 2441 bytes and the link count 2. After `rm README` (Figure 2.4) the name `r2` still shows the inode 2 and the size 2441 bytes, and the link count is 1. Only the link count changed (1, then 2, then 1); the inode number, the size and the content did not change, and `cat r2 | wc` (Figure 2.5) still reads the 2441 bytes of the old `README`. The values are collected in Table 2.
- The function `sys_unlink` first erases the entry of the name from the directory (it writes an empty entry with the inode number 0) and then decrements the count with the line `ip->nlink--;`, followed by `iupdate(ip)`. The function does not free any block itself. The data blocks are freed when the link count is 0 and no process keeps the file open: the function `iput` in `kernel/fs.c` then calls `itrunc`, which releases the blocks, and marks the inode as free. In the experiment the count fell only from 2 to 1, so nothing was freed.
- An xv6 inode (`struct dinode`) has only the fields `type`, `major`, `minor`, `nlink`, `size` and the array `addrs` of the block addresses (Table 3). It has no owner, no permission bits and no times, and the three types of Part 1 contain no symbolic link, so these features do not exist in xv6.



Figure 2.1: The beginning of user/mystat.c in the editor

nano user/mystat.c – the program from the task was completed; Figure 2.1 shows its beginning in the editor and the complete source is given below. The function `typename` converts the number of the type into a word with a `switch`: `"directory"` for `T_DIR`, `"file"` for `T_FILE`, `"device"` for `T_DEVICE` and `"?"` for any other value. The function `main` goes through the arguments (the loop starts from 1, because `argv[0]` is the name of the program) and calls `stat(argv[i], &st)`. If the call fails, the program prints a message on the standard error (descriptor 2) and continues with the next argument. Otherwise it prints the name, the type, the inode number, the number of links and the size, taken from the fields `type`, `ino`, `nlink` and `size` of `struct stat`. The numbers are converted to `int`, because the `printf` of xv6 prints them with `%d`. The program ends with `exit(0)`.

```
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  char *
6  typename(int t)
7  {
8    switch(t){
9    case T_DIR:
10     return "directory";
```

```
11    case T_FILE:
12      return "file";
13    case T_DEVICE:
14      return "device";
15    default:
16      return "?";
17    }
18  }
19
20  int
21  main(int argc, char *argv[])
22  {
23    struct stat st;
24
25    for(int i = 1; i < argc; i++){
26      if(stat(argv[i], &st) < 0){
27        fprintf(2, "mystat: cannot stat %s\n", argv[i]);
28        continue;
29      }
30
31      printf("%s: %s, inode %d, links %d, size %d bytes\n",
32          argv[i],
33          typename(st.type),
34          (int)st.ino,
35          (int)st.nlink,
36          (int)st.size);
37      }
38
39    exit(0);
40  }
```

Makefile – UPROGS – the line `$U/_mystat\` was added to the variable `UPROGS`, with a TAB at the beginning and nothing after the backslash, otherwise the program is not compiled and not copied into the xv6 file system. The list printed by `make clean` in Figure 3.2 contains `user/_mystat`, which confirms the line.

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 24, links 1, size 0 bytes
$
```

Figure 2.2: mystat applied to a file, a directory and a device

mystat README . console – the program prints one line for every argument. In Figure 2.2, `README` is a file with the inode 2, one link and 2441 bytes, `.` is a directory with the inode 1, one link and 1024 bytes, and `console` is a device with the inode 24, one link and size 0. The inode of `console` is 24 and not 23 as in Figure 1.1, because the new program `mystat` received an inode before it.

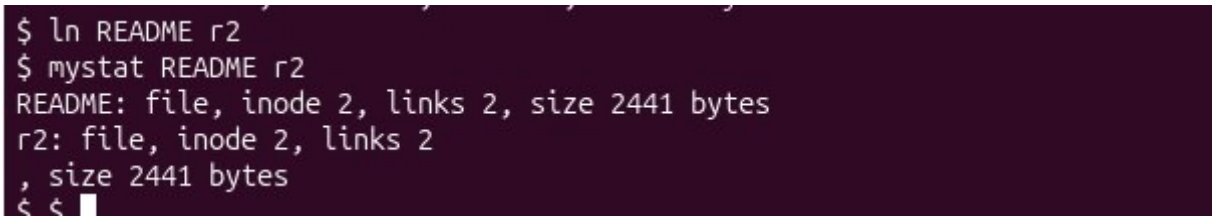


Figure 2.3: The link count after `ln README r2`

ln README r2 ; mystat README r2 – `ln` creates a second name for the same file. In Figure 2.3 both names show the inode 2, the size 2441 bytes and two links, so `r2` is not a copy but another entry that points to the same inode. The second line of the output is shown by the console in two parts (the text `, size 2441 bytes` is on the line below), but all the values are visible.

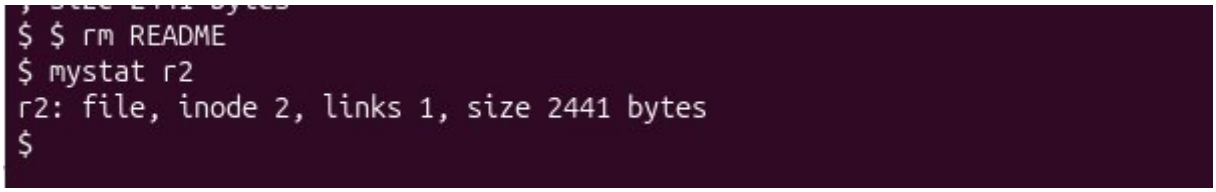


Figure 2.4: The link count after `rm README`

rm README ; mystat r2 – the name `README` is removed, and `r2` still shows the inode 2 and the size 2441 bytes, but only one link (Figure 2.4). The file was not deleted, because one name remained.



Figure 2.5: The content of the file is still readable through r2

cat r2 | wc – `cat` reads the file through the name `r2` and `wc` counts 48 lines, 336 words and 2441 bytes (Figure 2.5). The number of bytes is the same as the size shown by `mystat`, which proves that the data of `README` survived the removal of its first name.

Moment	**Name**	**Inode**	**Links**	**Size**
before `ln`	`README`	2	1	2441
after `ln README r2`	`README`	2	2	2441
after `ln README r2`	`r2`	2	2	2441
after `rm README`	`r2`	2	1	2441

Table 2: The values shown by `mystat` during the experiment with the hard link

Field	Meaning
`type`	file type: `T_DIR`, `T_FILE` or `T_DEVICE`
`major`, `minor`	device numbers, used only by the devices
`nlink`	number of names (links) of the inode
`size`	size of the file in bytes
`addrs`	addresses of the data blocks: `NDIRECT` direct blocks and one indirect block

Table 3: The fields of `struct dinode`

Table 2 shows that only the link count is changed by `ln` and `rm`, while the inode, the size and the data stay the same. Table 3 shows that the inode of xv6 is very small, because the names are not stored in it but in the directories, and the permissions and the owner do not exist.

Part 3: Measure the limits

Questions:

- What does `ls` show for `averyveryverylongname`, and which constant explains it?
- Compute the largest file xv6 can hold: 12 direct blocks plus one indirect block that holds block numbers of 4 bytes each, blocks of `BSIZE` bytes. Show the calculation.
- From the `nmeta` line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data?

Answers:

- The name `averyveryverylongname` has 21 characters, but the field for the name in a directory entry has only `DIRSIZ` = 14 bytes (Figure 3.1). Therefore the directory stores only the first 14 characters, `averyveryveryl`, and `ls` shows this shortened name; the rest of the name is lost, without any error message. The constant that explains it is `DIRSIZ`.
- The constants in Figure 3.1 are `BSIZE` = 1024, `NDIRECT` = 12 and `NINDIRECT` = `BSIZE / sizeof(uint)` = 1024/4 = 256. The indirect block holds 256 block numbers, so a file can have $12 + 256 = 268$ blocks, and the largest file has $268 \cdot 1024 = 274\,432$ bytes, that is, 268 KB.
- The line in Figure 3.2 is `nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000`. The whole disk has 2000 blocks. The log uses 31 blocks and the inodes use 13 blocks; the other metadata blocks are 1 boot block, 1 superblock and 1 block of the bitmap, so $1 + 1 + 31 + 13 + 1 = 47$ blocks are metadata and $2000 - 47 = 1953$ blocks are for data. The data blocks are shown in Table 4.

```
ubuntu@ubuntu:~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define N
INDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
ubuntu@ubuntu:~/xv6-riscv$
```

Figure 3.1: The constants of kernel/fs.h

grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h – searches the file for the four constants; the sign `\|` means "or" in the pattern of `grep`. Figure 3.1 shows `BSIZE` 1024 (line 5, the size of a block), `NDIRECT` 12 (line 26), `NINDIRECT (BSIZE / sizeof(uint))` (line 27) and `DIRSIZ` 14 (line 54).

```
ubuntu@ubuntu:~/xv6-riscv$ make clean ; make fs.img 2>&1 | grep nmeta
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_mystat user/_forktest user/_grep user/_init use
r/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs use
r/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forph
an user/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) bl
ocks 1953 total 2000
ubuntu@ubuntu:~/xv6-riscv$
```

Figure 3.2: The layout of the disk printed by `make`

make clean ; make fs.img 2>&1 | grep nmeta – `make clean` removes the old files (its lines are printed on the screen), and `make fs.img` builds the image of the file system with the program `mkfs`. The redirection `2>&1` joins the error output with the normal output, so that `grep` sees all the messages and prints only the line with `nmeta`. The result is in Figure 3.2: 47 blocks of metadata, 1953 blocks of data and 2000 blocks in total. With `BSIZE` = 1024 the disk has $2000 \cdot 1024 = 2\,048\,000$ bytes. The 13 blocks of inodes are enough for the inodes of 64 bytes (`struct dinode`), 16 in a block.

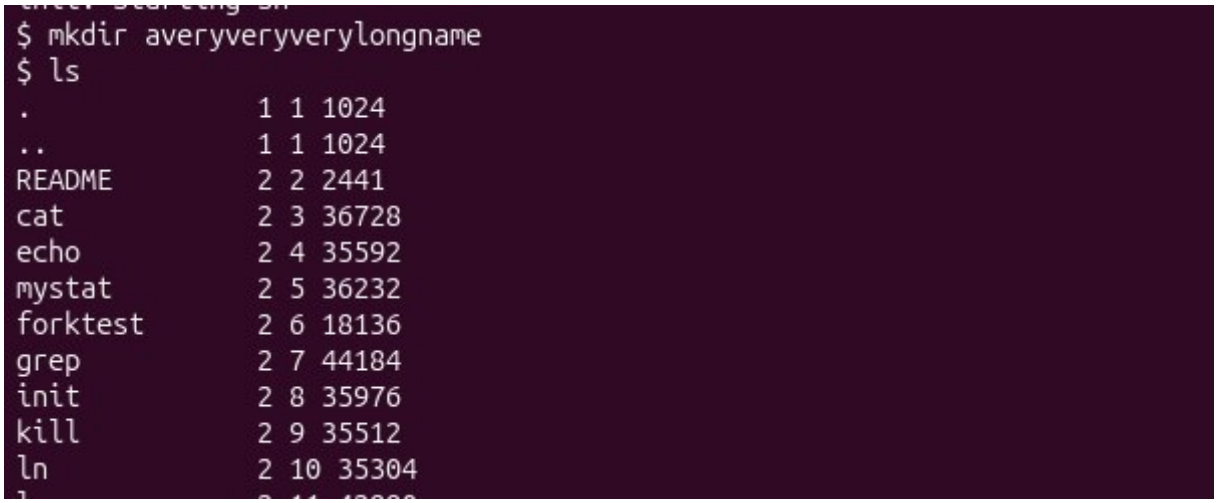


Figure 3.3: The command `mkdir averyveryverylongname` and the listing of `ls`

mkdir averyveryverylongname ; ls – the directory is created in the root directory, and `ls` lists the root again (Figure 3.3). In the part of the listing visible in Figure 3.3, the program `mystat` has the inode 5, which agrees with Part 2, and the new entry is stored in the root directory. Because the field for the name has only 14 bytes, the name of the new directory is cut to its first 14 characters, as explained in the answers.

Region of the disk	**Blocks**	**Remark**
boot block	1	metadata
superblock	1	metadata
log	31	metadata
inodes	13	metadata, 16 inodes in a block
bitmap	1	metadata
metadata (`nmeta`)	**47**	$1 + 1 + 31 + 13 + 1$
data	1953	$2000 - 47$
total	**2000**	$2000 \cdot 1024 = 2\,048\,000$ bytes

Table 4: The layout of the xv6 disk according to the `nmeta` line

Table 4 summarizes the measurements. Most of the disk (1953 of 2000 blocks) is used for data, and the limits that were measured, 14 characters for a name and 274432 bytes for a file, come directly from the constants of `kernel/fs.h`.

Part 4: Write find

nano user/find.c – the program `find <dir> <name>` prints every path under `<dir>` whose last component is `<name>`. It is built from the example of `user/ls.c`: the directory is opened with `open`, the entries are read one by one as `struct dirent`, the full path of every entry is built in a buffer and `stat` is called on it. The source is given below.

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"

char*
fmtname(char *path)
{
  static char buf[DIRSIZ+1];
  char *p;

  for(p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  if(strlen(p) >= DIRSIZ)
    return p;

  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), 0, DIRSIZ + 1 - strlen(p));

  return buf;
}

void
find(char *path, char *name)
{
  int fd;
  struct stat st;
  struct dirent de;
  char buf[512];
  char *p;

  if(stat(path, &st) < 0)
    return;

  if(strcmp(fmtname(path), name) == 0)
    printf("%s\n", path);

  if(st.type != T_DIR)
    return;

  fd = open(path, 0);
  if(fd < 0)
    return;

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while(read(fd, &de, sizeof(de)) == sizeof(de)){
```

```
52
53      if(de.inum == 0)
54        continue;
55
56      if(strcmp(de.name, ".") == 0 ||
57         strcmp(de.name, "..") == 0)
58        continue;
59
60      memmove(p, de.name, DIRSIZ);
61      p[DIRSIZ] = 0;
62
63      find(buf, name);
64    }
65
66    close(fd);
67  }
68
69  int
70  main(int argc, char *argv[])
71  {
72    if(argc != 3){
73      fprintf(2, "usage: find dir name\n");
74      exit(1);
75    }
76
77    find(argv[1], argv[2]);
78    exit(0);
79  }
```

fmtname – returns the last component of a path. The loop goes back from the end of the string to the last `/`, and `p++` moves to the first character of the name. If the name has `DIRSIZ` characters or more, it is returned as it is; otherwise it is copied into a static buffer and the rest of the buffer is filled with zeros. The padding is made with zeros and not with spaces as in `ls.c`, so that `strcmp` compares the real name.

find – the function calls `stat` on the path and prints the path if its last component is equal to the searched name. If the object is not a directory, the function returns. For a directory, the path is copied into `buf` and a `/` is added, then every entry is read with `read`. The entries with `inum == 0`, which are free slots, and the entries `.` and `..` are skipped, so that the recursion never goes up or loops. For all the other entries the name is appended to the path (the copy has `DIRSIZ` bytes, and the terminating zero is added by hand, because a name of 14 characters has none) and `find` is called again on the new path. At the end the directory is closed with `close`, because a process can have only 16 open files; since a directory stays open while its subdirectories are visited, very deep trees are limited by the same number and by the size of the stack.

main – checks that exactly two arguments were given. Otherwise it prints `usage: find dir name` on the standard error and ends with the code 1. The line `$U/_find\` must also be present in `UPROGS`, and the program works in Figure 4.1.



Figure 4.1: Testing find on a fresh disk

mkdir a ; mkdir a/b ; echo x > a/b/r2 ; echo y > a/r2 ; ln cat r2 – prepares the test tree: the directories `a` and `a/b`, two different files named `r2` with the content `x` and `y`, and a hard link `r2` to the program `cat` in the root directory (Figure 4.1).

find . r2 – prints the three paths `./a/b/r2`, `./a/r2` and `./r2`, that is, all the entries named `r2` in the tree. The last one is the link to `cat`.

find a b – prints only `a/b`, because it is the only entry under `a` whose last component is `b`; the file `a/b/r2` is not printed, since only the last component of the path is compared.

Command	**Output**	**Reason**
`find . r2`	`./a/b/r2`, `./a/r2`, `./r2`	three entries have the last component `r2`: two files created with `echo` and the link to `cat`
`find a b`	`a/b`	only the directory `b` has this name; `a/b/r2` ends with `r2`

Table 5: The results of the tests of `find`

Table 5 shows that the program visits every directory of the tree exactly once and compares only the last component of every path with the searched name.

Conclusion

The laboratory work looked inside the file system of xv6 and showed that it implements, in a few hundred lines, the same ideas as the file systems of Linux. In the first part, the output of `ls` was decoded with `kernel/stat.h`: the types 1, 2 and 3 are the directory, the file and the device, the root directory has the inode 1 and is its own parent, and a directory entry takes 16 bytes, so an empty directory with the entries `.` and `..` has the size 32. In the second part, the program `mystat`, written with the system call `stat`, showed that `ln` only adds a name to the same inode: the link count changed from 1 to 2 and back to 1 after `rm README`, while the inode 2, the size 2441 and the content of the file, counted by `wc`, stayed the same. The inode of xv6 contains only the type, the device numbers, the link count, the size and the block addresses, so there are no owners, permissions or symbolic links.

In the third part, the constants of `kernel/fs.h` were measured: the block of 1024 bytes, the 14 characters of a name, and the 12 direct and 256 indirect blocks, which limit a file to 274432 bytes. The `nmeta` line showed that the disk has 2000 blocks, of which 47 hold metadata (boot, superblock, 31 blocks of log, 13 of inodes and the bitmap) and 1953 are left for data. In the last part, the program `find` walked through the tree recursively with the system calls `open`, `read`, `stat` and `close`, skipped the free entries and the entries `.` and `..`, and found the three files `r2` and the directory `b`. Overall, the laboratory work gave a practical understanding of inodes, directories and links, of the layout of the disk, and of the way a program reads a directory, and all the objectives of the work were achieved.

References

1. *Gabriela Pricop. os-2026-GabrielaPricop: laboratory works of the Operating Systems course, GitHub repository*
   `https://github.com/gabriela-pricop/os-2026-GabrielaPricop`
2. *Xv6, a simple Unix-like teaching operating system: the book and the source code (MIT 6.1810)*
   `https://pdos.csail.mit.edu/6.828/xv6`
3. *Lab: Xv6 and Unix utilities (MIT 6.1810), the find exercise*
   `https://pdos.csail.mit.edu/6.828/2023/labs/util.html`
4. *unlink(2) – delete a name and possibly the file it refers to, Linux manual page*
   `https://manpages.debian.org/testing/manpages-dev/unlink.2.en.html`